Introduction

Object detection is an important part of computer vision, with applications in areas like autonomous driving, assistive technologies, and robotics (Redmon et al., 2015). You Only Look Once (YOLO) algorithm presents a significant advancement in this field by using a fast method for object detection in images. Unlike previous multi-stage approaches, YOLO uses a single pass through a neural network that simultaneously predicts bounding boxes and class probabilities directly from full images. This allows YOLO to be a significantly faster object detection algorithm than multi-stage approaches.

This approach is highly relevant to machine learning because it demonstrates how deep learning models can be designed to perform complex tasks such as forming bounding boxes and assigning confidence intervals, simultaniously in a unified and computationally efficient manner. Many earlier algorithmic frameworks for object detection (such as R-CNN) operate in multiple steps, with initial pass generating proposals followed by second pass for classification. In contrast, YOLO eliminates these intermediate steps, trading computational time for accuracy, allowing real-time application (Kundu, 2026).

The impact of YOLO is in real-world applications where speed and accuracy are critical. It is used in autonomous vehicles to replace specialized sensors, in assistive devices to provide real-time scene understanding, and in robotics for dynamic interaction with environments (Redmon et al., 2015; Wang et al., 2022). Over several years of innovation, optimization, and incremental improvements, the latest version of YOLO, currently YOLO v26, released as of January this year by Ultralytics, is one of the front runners for object detection algorithms in the industry (Ultralytics, 2026).

How It Works

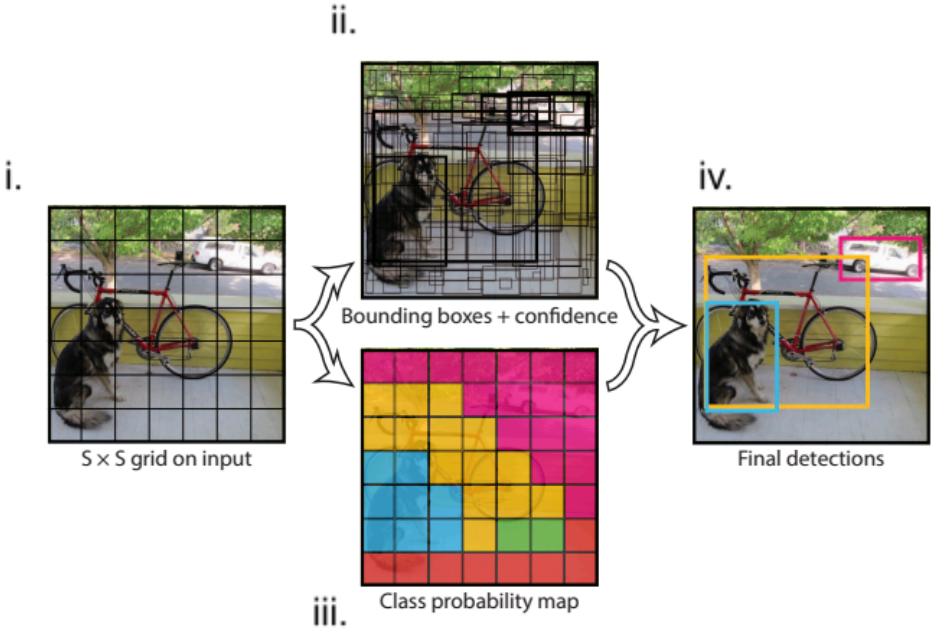


Figure 1. Depicts a visual representation of how the algorithm pipeline processes an image into 3 general steps. i.) Image is divided into an S × S grid, ii,iii.) recursively generate B bounding boxes per grid cell and the confidence of each bounding box, while simultaneously assigning C class probabilities to each grid square. These predictions are encoded as an S × S × (B ∗ 5 + C) tensor. iv.) Detections are then filtered based on model confidence and output as a final prediction with corresponding object bounding boxes. (figure derived from Redmon et al., 2015)

Before discussing the YOLO pipeline, a brief explanation of Convolutional Neural Networks (CNNs). Two key components that make CNNs are convolutional and fully connected layers.

Convolutional layers are a form of feature extraction. By taking a smaller window (learnable filters or kernels), and swiping it through the large image. Every time a filter is passed through the image, a formula called a convolution is applied. Thus, we end with a slightly smaller filtered image, which in some way summarizes the previous one and detects patterns such as edges, textures, and more complex shapes. As data passes through multiple convolutional layers, the network learns increasingly abstract and high-level features. (Serrano, 2021)

Fully connected layers (neural networks), typically used near the end of the network is for decision-making. In these layers, every neuron is connected to every other neuron in the previous layer, allowing the model to combine all extracted features and produce final outputs, such as class probabilities or detections. Together, convolutional and fully connected layers enable CNNs to both understand visual patterns and make accurate predictions based on them (Serrano, 2021).

The most basic form of YOLO uses a single convolutional network that predicts multiple bounding boxes and class probabilities for each box at the same time. YOLO resizes the input images into an S X S grid, runs the convolutional network on the image to predict B bounding boxes, the confidence for those boxes, and C class probabilities, creating an S × S × (B ∗ 5 + C) tensor. YOLO then filters the resulting detections by the model's confidence to give the final prediction.

The earliest form of YOLO used a network that had 24 convolutional layers followed by 2 fully connected layers, creating a 7x7x30 tensor. (Redmon et al., 2015). Later iterations use the same single CNN concept, however have changes in the CNN architecture, such as CSPNet (Wang et al. 2021) or in how bounding boxes are organized, ie, anchored boxes (Redmon and Farhadi, 2016).

YOLO models are trained using annotated datasets consisting of many images, where each image may contain multiple objects. Each object is associated with a ground-truth label, which represent the correct bounding box coordinates and class information. Ground-truth data is pre-labeled information that the model trains on and aims to predict.

During training, YOLO divides an image into a grid, and each grid cell predicts multiple bounding boxes. For each object, the model assigns one predictor to be "responsible" based on

which predicted bounding box has the highest Intersection over Union (IoU) with the corresponding ground-truth box.(IoU, which will be explained further in performance, is a metric used to evaluate how well the predicted bounding box overlaps with the ground-truth.)

Use Cases & Performance

YOLO performs best in scenarios where real-time detection is required, and slight reductions in accuracy are acceptable, such as tracking and surveillance or computer/robotic vision. A specific example includes self-driving cars (Redmon et al., 2015), quality assurance in product lines, and automation for traditionally manual labour; a more biologically focused example would be live surgical and ultrasound videos, which can be used as an aid to help healthcare professionals identify and annotate anomalies that would have been otherwise overlooked and increase throughput (Wang et al. 2025).

Performance for image recognition algorithms is measured by Intersection over Union (IoU) and by Average Precision (AP).

IoU is calculated by taking the intersecting area between the predicted and the ground truth bounding boxes for the same object, referred to as“Overlap”. Then the total area covered by the two bounding boxes is calculated, referred to as the “Union”. By dividing the Overlap over the Union, you get the ratio referred to as IoU (Kundu, 2026). An IoU of 1 would indicate that the predicted bounding box is equal to the ground truth, thus a perfect prediction; an IOU of 0 would refer to a failed prediction.

AP or AUPRC is another metric used for determining the precision-recall trade-off. A common figure used in comparing object detection methods, mAP50-95 refers to the average of mAP calculated at IoU thresholds from 0.50 to 0.95 in steps of 0.05, and epoch refers to the number of cycles/iterations

YOLO’s main advantages are its speed, as it performs object detection in a single pass through the network, making it significantly faster than two-stage detectors such as R-CNN. However, this speed comes with trade-offs. YOLO can struggle with detecting small objects or objects in tight clusters like flocks of birds (Redmon et al., 2015).

Code Implementation (Python using demo provided by Ultralytics)

```
from ultralytics import YOLO

# Load a pretrained YOLO model, model = YOLO("yolo26n.yaml") where .yaml containing parameter ‘mode’ and optional arguments ‘task’ and ‘args’
model = YOLO("yolo26n.pt")

# Train the model using the 'coco8.yaml' dataset for 3 epochs (cycles)
```

```
results = model.train(data="coco8.yaml", epochs=3)

# Evaluate the model's performance on the validation set
results = model.val()

# Perform object detection on an image using the model
results = model("https://ultralytics.com/images/bus.jpg")
```

“””

https://image2url.com/r2/default/images/1775513604070-d3ac0bf0-da75-475e-8a09-7e1f1cba6af7.jpg

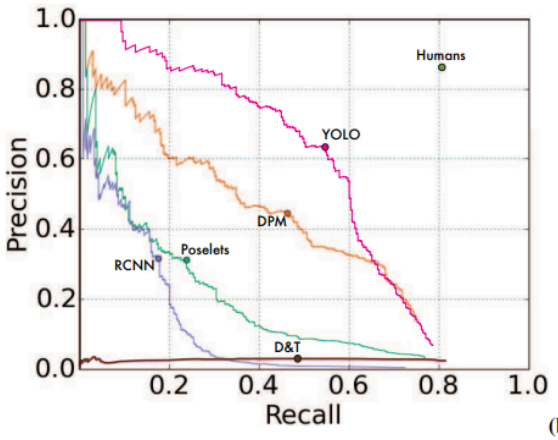
image 1/1 C:\Users\Delwar\Desktop\Humber\WINTER SEMESTER\BINF-5507 Machine Learning AI Bioinformatics\Labs and Assignments\Capstone Project Report and Presentation\bus.jpg: 640x480 4 persons, 1 bus, 320.0ms
Speed: 7.3ms preprocess, 320.0ms inference, 2.1ms postprocess per image at shape (1, 3, 640, 480)

“””

```
# Export the model to ONNX format, an open-source format designed to represent machine learning models
success = model.export(format="onnx")
```

Comparison / Visualization

YOLO's many iterations are often compared with other popular image recognition algorithms (such as RCNN) as well as previous YOLO versions using the AP to epoch curve. Another common comparison is AP versus Time (ms or FPS) to showcase the speed of YOLO against other algorithms, with AP being the same or better than previous versions or algorithms.



(a) Picasso Dataset precision-recall curves.

	VOC 2007	Picasso		People-Art
	AP	AP	Best F_1	AP
YOLO	**59.2**	**53.3**	**0.590**	**45**
R-CNN	54.2	10.4	0.226	26
DPM	43.2	37.8	0.458	32
Poselets [2]	36.5	17.8	0.271	
D&T [4]	-	1.9	0.051	

(b) Quantitative results on the VOC 2007, Picasso, and People-Art Datasets. The Picasso Dataset evaluates on both AP and best F_1 score.

Figure 2. PRC plotting curve of all 4 detection methods using the 'Picasso dataset'. Comparing 4 object detection algorithms using various datasets and corresponding AP (AUPRC) scores

This PRC plot with complimentary table shown above (Figure 2) from the original YOLO paper shows the performance of YOLO in comparison to other detection methods: R-CNN, DPM, Poselets, and D&T.

According to the paper, VOC 2007 detection AP for 'person'; all models are trained only on VOC 2007 data. On Picasso models are trained on VOC 2012 while on People-Art they are trained on VOC 2010.(Redmon et al., 2015)

YOLO has good performance on VOC 2007, and its AP drops less than other methods when applied to artwork. Like DPM, YOLO models the size and shape of objects, as well as the relationships between objects and their orientation. Artwork and natural images are similar in terms of the size and shape of objects; YOLO can still predict good bounding boxes and detections unlike other methods like R-CNN.

References

- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779–788. https://doi.org/10.1109/cvpr.2016.91
- Redmon, J., & Farhadi, A. (2017). Yolo9000: Better, faster, stronger. 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 6517–6525. https://doi.org/10.1109/cvpr.2017.690
- Ultralytics (2026) Ultralytics YOLO26, url:https://docs.ultralytics.com/models/yolo26/
- Rohit Kundu (2026) What is YOLO architecture and how does it work? Learn about different YOLO algorithm versions and start training your own YOLO object detection models. url:https://www.v7labs.com/blog/yolo-object-detection
- Wang, C.-Y., Bochkovskiy, A., & Liao, H.-Y. M. (2023). Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 7464–7475. https://doi.org/10.1109/cvpr52729.2023.00721
- Yolo algorithm for object detection explained [+examples]. YOLO Algorithm for Object Detection Explained [+Examples]. (n.d.). https://www.v7labs.com/blog/yolo-object-detection
- Demo derived from https://docs.ultralytics.com/usage/python/
- Wang, S., Zhao, Z.-A., Chen, Y., Mao, Y.-J., & Cheung, J. C.-W. (2025). Enhancing Thyroid Nodule Detection in Ultrasound Images: A Novel YOLOv8 Architecture with a C2fA Module and Optimized Loss Functions. Technologies, 13(1), 28. https://doi.org/10.3390/technologies13010028